

Type Checking (P.341)

- A compiler has to do semantic checks in addition to syntactic checks.
- Semantic Checks
 - Static – done during compilation
 - Dynamic – done during run-time
- *Type checking* is one of these static checking operations.
 - we may not do all type checking at compile-time.
 - Some systems also use dynamic type checking too.



Type system (P.344)

- A *type system* is a collection of rules for assigning type expressions to the parts of a program.
- A *type checker* implements a type system.
- A *sound* type system eliminates run-time type checking for type errors.
- A programming language is *strongly-typed*, if every program its compiler accepts will execute without type errors.
 - In practice, some of type checking operations are done at run-time (so, most of the programming languages are not strongly-typed).
 - Ex: `int x[100]; ... x[i]` → most of the compilers cannot guarantee that `i` will be between 0 and 99

Type Expression (P.345)



- The type of a language construct is denoted by a *type expression*.
- A *type expression* can be:
 - A basic type
 - a primitive data type such as *integer*, *real*, *char*, *boolean*, ...
 - *type-error* to signal a type error
 - *void*: no type
 - A type name
 - a name can be used to denote a type expression.

Type Expression (cont.)

- A type constructor applies to other type expressions.
 - arrays: If T is a type expression, then $array(I, T)$ is a type expression where I denotes index range. Ex: $array(0..99, int)$
 - products: If T_1 and T_2 are type expressions, then their cartesian product $T_1 \times T_2$ is a type expression. Ex: $int \times int$
 - pointers: If T is a type expression, then $pointer(T)$ is a type expression. Ex: $pointer(int)$
 - functions: We may treat functions in a programming language as mapping from a domain type D to a range type R . So, the type of a function can be denoted by the type expression $D \rightarrow R$ where D and R are type expressions. Ex: $int \rightarrow int$ represents the type of a function which takes an int value as parameter, and its return type is also int .

A Simple Type Checking System (P.348)

The type checker is a translation scheme that synthesizes the type of each expression from the types of its sub-expressions.

$$P \rightarrow D; E$$
$$D \rightarrow D; D$$
$$D \rightarrow \text{id}: T \quad \{ \text{addtype}(\text{id.entry}, T.\text{type}) \}$$
$$T \rightarrow \text{char} \quad \{ T.\text{type} = \text{char} \}$$
$$T \rightarrow \text{int} \quad \{ T.\text{type} = \text{int} \}$$
$$T \rightarrow \text{real} \quad \{ T.\text{type} = \text{real} \}$$
$$T \rightarrow \uparrow T_1 \quad \{ T.\text{type} = \text{pointer}(T_1.\text{type}) \}$$
$$T \rightarrow \text{array}[\text{intnum}] \text{ of } T_1 \\ \{ T.\text{type} = \text{array}(0..\text{intnum.val}, T_1.\text{type}) \}$$

Type Checking of Expressions (P.35)



$E \rightarrow \text{id} \quad \{ E.\text{type} = \text{lookup}(\text{id}.\text{entry}) \}$

$E \rightarrow \text{charliteral} \quad \{ E.\text{type} = \text{char} \}$

$E \rightarrow \text{intliteral} \quad \{ E.\text{type} = \text{int} \}$

$E \rightarrow \text{realliteral} \quad \{ E.\text{type} = \text{real} \}$

$E \rightarrow E_1 + E_2 \quad \{ \text{if } (E_1.\text{type} = \text{int} \text{ and } E_2.\text{type} = \text{int}) \text{ then } E.\text{type} = \text{int}$
 $\text{else if } (E_1.\text{type} = \text{int} \text{ and } E_2.\text{type} = \text{real}) \text{ then } E.\text{type} = \text{real}$
 $\text{else if } (E_1.\text{type} = \text{real} \text{ and } E_2.\text{type} = \text{int}) \text{ then } E.\text{type} = \text{real}$
 $\text{else if } (E_1.\text{type} = \text{real} \text{ and } E_2.\text{type} = \text{real}) \text{ then } E.\text{type} = \text{real}$
 $\text{else } E.\text{type} = \text{type-error} \}$

$E \rightarrow E_1 [E_2] \quad \{ \text{if } (E_2.\text{type} = \text{int} \text{ and } E_1.\text{type} = \text{array}(s, t)) \text{ then } E.\text{type} = t$
 $\text{else } E.\text{type} = \text{type-error} \}$

$E \rightarrow \uparrow E_1 \quad \{ \text{if } (t = \text{pointer}(E_1.\text{type})) \text{ then } E.\text{type} = t$
 $\text{else } E.\text{type} = \text{type-error} \}$

$S \rightarrow id = E$

{ if ($id.type=E.type$ then $S.type=void$
else $S.type=type-error$ }

$$S \rightarrow \text{if } E \text{ then } S_1 \quad \{ \text{if } (E.\text{type}=\text{boolean} \text{ then } S.\text{type}=S_1.\text{type} \\ \text{else } S.\text{type}=\text{type-error} \}$$
$$S \rightarrow \text{while } E \text{ do } S_1 \quad \{ \text{if } (E.\text{type}=\text{boolean} \text{ then } S.\text{type}=S_1.\text{type} \\ \text{else } S.\text{type}=\text{type-error}) \}$$
$$S \rightarrow S_1; S_2 \quad \{ \text{if } (S_1.\text{type}=\text{void} \text{ and } S_2.\text{type}=\text{void} \text{ then } S.\text{type}=\text{void} \\ \text{else } S.\text{type}=\text{type-error} \}$$

Type Checking of Functions (P.352)


$$E \rightarrow E_1 (E_2) \quad \{ \text{if } (E_2.\text{type}=s \text{ and } E_1.\text{type}=s \rightarrow t) \text{ then} \\ E.\text{type}=t \\ \text{else } E.\text{type}=\text{type-error} \}$$

Ex: `int f(double x, char y) { ... }`

`f: double x char  $\rightarrow$  int`



argument types

return type

Structural Equivalence of Type Expressions (P.353)



- How do we know that two type expressions are equal?
- As long as type expressions are built from basic types (no type names), we may use structural equivalence between two type expressions

Structural Equivalence Algorithm (**sequiv**):

```
if (s and t are same basic types) then return true
else if (s=array(s1,s2) and t=array(t1,t2)) then return (sequiv(s1,t1) and sequiv(s2,t2))
else if (s = s1 x s2 and t = t1 x t2) then return (sequiv(s1,t1) and sequiv(s2,t2))
else if (s=pointer(s1) and t=pointer(t1)) then return (sequiv(s1,t1))
else if (s = s1 → s2 and t = t1 → t2) then return (sequiv(s1,t1) and sequiv(s2,t2))
else return false
```

Names for Type Expressions (P.356)



- In some programming languages, we give a name to a type expression, and we use that name as a type expression afterwards.

type link = $\uparrow$ cell;

var p,q : link;

var r,s : $\uparrow$ cell

p,q,r,s have same types ?

Under name equivalence, p and r do not have the same type, however, under structural equivalence, all variables p, q, r and s have the same type.

- How do we treat type names?
 - Get equivalent type expression for a type name (then use structural equivalence), or
 - Treat a type name as a basic type.

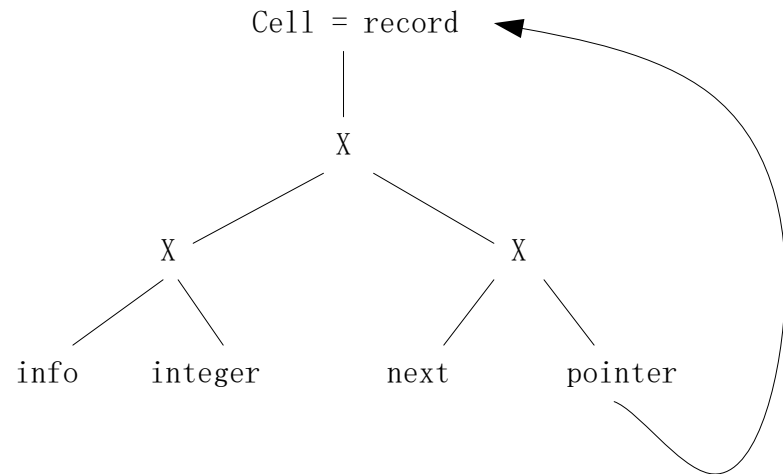
Cycles in Type Expressions (P.358)



```
type link = ↑ cell;  
type cell = record
```

```
  info : int,  
  next : link
```

```
end;
```



- The type name `link` is defined in terms of `cell` and that `cell` is defined in terms of `link`, so their definitions are recursive.
- We cannot use structural equivalence if there are cycles in type expressions.
- We have to treat type names as basic types.
 - but this means that the type expression `link` is different than the type expression `↑ cell`.

Type Coercions (P.359)

$x + y$

what is the type of this expression (int or double)?

- What kind of codes we have to produce, if the type of x is double and the type of y is int?

inttoreal $t1, y$

real+ $t2, x, t1$ (addf \$f3, \$f1, \$f2) $\uparrow$
 $t2 \quad x \quad t1$

Overloading (P.361)



- Expressions might have a multitude of types.

$$\begin{aligned} E &\rightarrow \text{id} && \{E.types = \text{lookup}(\text{id}.entry)\} \\ E &\rightarrow E_1 + E_2 && \{E.types = \{t \mid \text{there exists a type in } E_1.types \text{ and } E_2.types \text{ simultaneously}\}\} \\ E &\rightarrow E_1(E_2) && \{E.types = \{t \mid \text{there exists a } s \text{ in } E_2.types \text{ such that } s \rightarrow t \text{ belongs to } E_1.types\}\} \end{aligned}$$

- An overloaded symbol is one that has different meanings depending on its context.
 - The type of E is determined by looking at the possible types of E_1 and E_2 .
- Type error will occur when $E.types$ becomes empty for some expression.

1、对于输入的表达式 $(4*7+1)*2$ ，根据下表的语法制导定义建立一棵带注释的分析树。

val :表示非终结符的整数值, 综合属性, $lexval$ 是单词 $digit$ 的属性
语法制导定义如下:

产生式	语义规则
$L \rightarrow E$	$print(E.val)$
$E \rightarrow E^1 + T$	$E.val := E^1.val + T.val$
$E \rightarrow T$	$E.val := T.val$
$T \rightarrow T^1 * F$	$T.val := T^1.val * F.val$
$T \rightarrow F$	$T.val := F.val$
$F \rightarrow (E)$	$F.val := E.val$
$F \rightarrow digit$	$F.val := digit.lexval$

2、请按语法制导的定义，将后缀表达式翻译成中缀表达式。
注意，不允许出现冗余括号，后续表达式的文法如下：

$E \rightarrow EE+$

$E \rightarrow EE^*$

$E \rightarrow id$

3、下面文法产生的表达式是对整型和实型常数应用算符+形成的。当两个整数相加时,结果为整数, 否则为实数。

$$E \rightarrow TR$$
$$R \rightarrow + TR | \epsilon$$
$$T \rightarrow \text{num.num} \mid \text{num}$$

- a) 给出语法制导定义确定每个子表达式的类型。
- b) 把表达式翻译成前缀形式, 并且决定类型。试用一元运算符 `inttoreal` 把整型值转换为相等的实型值, 以使得前缀表达式中两个运算对象是同类型的。